

Quyidagi tushunchalar neural network, ya'ni sun'iy neyron tarmoqni o'qitish va uning natijalarini baholashdagi eng asosiy jarayonlar hisoblanadi. Ayniqsa YOLO kabi object detection modellarida forward pass, backward pass, batch, epoch, validation loss, precision, recall va confusion matrix bir-biri bilan chambarchas bog'liq ishlaydi.

1. Forward pass va backward pass farqi

Forward pass nima?

Forward pass — ma'lumotning modelga kirib, qatlamlardan oldinga qarab o'tishi va yakuniy prediction, ya'ni bashorat hosil qilishi jarayoni.

Forward — oldinga yo'nalish degan ma'noni beradi.

Pass — ma'lumotning model qatlamlari orqali o'tishi.

Rasm modelga berilganda quyidagi jarayon sodir bo'ladi:

Rasm

- model qatlamlari
- xususiyatlarni ajratish
- prediction
- loss hisoblash

YOLO modelida forward pass davomida rasm:

Input image

- Backbone
- Neck
- Detection Head
- Bounding box + class + confidence

tartibida qayta ishlanadi.

Input image — modelga kiruvchi rasm.

Backbone — rasmdan muhim xususiyatlarni ajratib oluvchi qism.

Neck — turli o'lchamdagi feature maplarni birlashtiruvchi qism.

Detection Head — bounding box va class bashoratini chiqaruvchi yakuniy qism.

Oddiy matematik misol

Bitta sodda neyron quyidagi hisobni bajaradi:

$$z=wx+bz = wx + bz=wx+b$$

Bu yerda:

- xxx — kiruvchi qiymat;
- www — weight, ya'ni model o'rganadigan og'irlik;
- bbb — bias, ya'ni qo'shimcha siljitish qiymati;
- zzz — oraliq natija.

Masalan:

$$x=2x = 2x=2 \quad w=0.5w = 0.5w=0.5 \quad b=1b = 1b=1$$

bo'lsa:

$$z=0.5 \times 2 + 1 = 2z = 0.5 \times 2 + 1 = 2z=0.5 \times 2 + 1 = 2$$

Keyin activation function ishlatilishi mumkin.

Activation function — neyron natijasiga nochiziqlilik qo'shuvchi funksiya.

Masalan, ReLU:

$$ReLU(z)=\max(0,z)ReLU(z) = \max(0,z)ReLU(z)=\max(0,z)$$

$z=2z=2z=2$ bo'lsa:

$$\text{ReLU}(2)=2\text{ReLU}(2)=2\text{ReLU}(2)=2$$

Mana shu hisoblarning kirishdan chiqishgacha bajarilishi forward pass hisoblanadi.

YOLO modelida forward pass natijasi

YOLO forward pass vaqtida har bir topilgan obyekt uchun quyidagilarni chiqaradi:

- bounding box koordinatalari;
- class bashorati;
- confidence qiymati.

Masalan:

class: car

confidence: 0.91

box: [120, 80, 450, 330]

Bu natijada:

- model obyektini car, ya'ni avtomobil deb topgan;
- confidence 0.91, ya'ni model 91 foiz ishonch bildirgan;
- box obyektning rasmdagi joylashuvini ko'rsatgan.

Training jarayonida model predictioni haqiqiy annotation bilan taqqoslanadi.

Masalan, haqiqiy class:

truck

model predictioni:

car

bo'lsa, xato mavjud bo'ladi.

Bounding box noto'g'ri joylashgan bo'lsa ham xato hisoblanadi.

Bu xatolar loss function yordamida o'lchanadi.

Loss function — model xatosini bitta son bilan ifodalovchi funksiya.

YOLO modelida loss bir necha qismdan tashkil topishi mumkin:

- box loss;
- class loss;
- DFL loss.

Box loss

Box loss — prediction qilingan bounding box bilan haqiqiy bounding box orasidagi farqni o'lchaydi.

Box obyektning noto'g'ri o'rab olsa, box loss katta bo'ladi.

Class loss

Class loss — model bashorat qilgan class bilan haqiqiy class orasidagi farqni o'lchaydi.

Masalan, haqiqiy class **dog**, prediction esa **cat** bo'lsa, class loss ortadi.

DFL loss

DFL — Distribution Focal Loss — bounding box chegaralarini aniqroq bashorat qilishga yordam beruvchi loss turi.

Distribution — taqsimot.

Focal — model e'tiborini murakkab yoki noto'g'ri predictionlarga ko'proq qaratish.

Forward pass oxirida umumiy loss taxminan quyidagicha hosil bo'ladi:

$$Loss = BoxLoss + ClassLoss + DFLoss$$

Aniq formula model versiyasi va sozlamalariga bog'liq bo'ladi.

Backward pass nima?

Backward pass — forward passda hisoblangan loss asosida model weightlarini qanday o'zgartirish kerakligini aniqlash jarayoni.

Backward — orqaga yo'nalish.

Forward pass ma'lumotni kirishdan chiqishga olib boradi. Backward pass esa lossdan boshlab modelning oldingi qatlamlariga qarab orqaga yuradi.

Jarayon:

Loss

- gradientlarni hisoblash
- qatlamlar bo'ylab orqaga tarqatish
- weightlarni yangilash

Bu jarayon **backpropagation** deyiladi.

Backpropagation — xatoni tarmoq bo'ylab orqaga tarqatish.

Gradient nima?

Gradient — weight ozgina o'zgarganda loss qaysi yo'nalishda va qancha o'zgarishini ko'rsatuvchi qiymat.

Soddalashtirilgan ko'rinishda:

$$\frac{\partial \text{Loss}}{\partial w} = \frac{\partial \text{Loss}}{\partial w}$$

bu ifoda lossning w weightga nisbatan hosilasini bildiradi.

Agar gradient musbat bo'lsa, weightni kamaytirish lossni kamaytirishi mumkin.

Agar gradient manfiy bo'lsa, weightni oshirish lossni kamaytirishi mumkin.

Weight qanday yangilanadi?

Gradient descent asosida weight quyidagicha yangilanadi:

$$w_{\text{new}} = w_{\text{old}} - \text{learning rate} \times \text{gradient}$$

Masalan:

$$\begin{aligned} w_{\text{old}} &= 0.5 \\ \text{learning rate} &= 0.01 \\ \text{gradient} &= 2 \end{aligned}$$

bo'lsa:

$$\begin{aligned} w_{\text{new}} &= 0.5 - 0.01 \times 2 \\ w_{\text{new}} &= 0.48 \end{aligned}$$

Model keyingi forward passda yangi **0.48** weight bilan hisoblaydi.

Learning rate — weightlar qanchalik katta qadam bilan yangilanishini belgilovchi qiymat.

Learning rate juda katta bo'lsa, model optimal nuqtadan o'tib ketishi mumkin.

Learning rate juda kichik bo'lsa, training juda sekinlashadi.

Optimizer vazifasi

Optimizer — gradientlardan foydalanib model weightlarini yangilaydigan algoritm.

Ko'p ishlatiladigan optimizerlar:

- SGD;
- Adam;
- AdamW.

SGD – **Stochastic Gradient Descent** — gradient asosida weightlarni bosqichma-bosqich yangilovchi algoritm.

Adam — gradientning oldingi qiymatlari va o'zgarish tezligini hisobga oluvchi optimizer.

AdamW — weight decay alohida qo'llanadigan Adam varianti.

Weight decay — model weightlarining haddan tashqari kattalashishini cheklashga yordam beruvchi regularization usuli.

Forward va backward pass ketma-ketligi

Bitta training iteration davomida quyidagilar bajariladi:

1. Batch modelga beriladi
2. Forward pass bajariladi
3. Prediction olinadi

4. Loss hisoblanadi
5. Gradientlar tozalanadi
6. Backward pass bajariladi
7. Optimizer weightlarni yangilaydi

PyTorch'da soddalashtirilgan ko'rinish:

```
optimizer.zero_grad()
```

```
predictions = model(images)
```

```
loss = loss_function(predictions, targets)
```

```
loss.backward()
```

```
optimizer.step()
```

Bu yerda:

```
optimizer.zero_grad()
```

oldingi gradientlarni tozalaydi.

```
predictions = model(images)
```

forward pass bajaradi.

```
loss.backward()
```

backward pass bajaradi.

```
optimizer.step()
```

weightlarni yangilaydi.

Asosiy farq

Xususiyat	Forward pass	Backward pass
Yo'nalish	Kirishdan chiqishga	Lossdan oldingi qatlamlarga

Asosiy vazifa	Prediction olish	Gradient hisoblash
Weight o'zgaradimi?	Yo'q	Optimizer orqali o'zgaradi
Loss hisoblanadimi?	Ha	Hisoblangan loss ishlatiladi
Trainingda ishlatiladimi?	Ha	Ha
Oddiy predictionda ishlatiladimi?	Ha	Yo'q

Model train qilinayotganda forward va backward pass ikkalasi ham bajariladi.

Model faqat prediction qilayotganda faqat forward pass bajariladi. Chunki yangi rasmda model weightlarini o'zgartirish talab qilinmaydi.

2. Batch size va epoch farqi

Batch size nima?

Batch size — modelga bir vaqtning o'zida beriladigan rasmlar soni.

Batch — ma'lumotlarning bitta kichik guruhi.

Masalan, train datasetda 1000 ta rasm mavjud bo'lsin.

Agar:

`batch size = 10`

bo'lsa, model bir vaqtning o'zida 10 ta rasmni qayta ishlaydi.

1000 ta rasm quyidagi guruhlariga bo'linadi:

1-batch: 1-10-rasmlar

2-batch: 11-20-rasmlar

3-batch: 21-30-rasmlar

...

100-batch: 991-1000-rasmlar

Har bir batch uchun:

1. forward pass bajariladi;
2. loss hisoblanadi;
3. backward pass bajariladi;
4. weightlar yangilanadi.

Shuning uchun 1000 ta rasm va batch size 10 bo'lsa, bitta epochda weightlar taxminan 100 marta yangilanadi.

Epoch nima?

Epoch — modelning butun train datasetini bir marta to'liq ko'rib chiqishi.

Masalan, datasetda 1000 ta rasm mavjud bo'lsa:

1 epoch = 1000 ta rasmning barchasi bir marta ishlatiladi

Agar:

epochs = 5

bo'lsa, model train datasetdagi rasmlarni 5 marta qayta ko'rib chiqadi.

Bu 5000 ta yangi rasm mavjud degani emas. Ayni 1000 ta rasm bir nechta epoch davomida qayta ishlatiladi. Augmentation mavjud bo'lsa, bir rasm har epochda biroz boshqacha ko'rinishda modelga berilishi mumkin.

Iteration yoki step nima?

Iteration yoki **step** — bitta batch bo'yicha forward pass, backward pass va weight yangilanishining bir marta bajarilishi.

Formula:

$$\text{Steps per epoch} = \left\lceil \frac{\text{Dataset size}}{\text{Batch size}} \right\rceil$$

Ceiling — natijani yuqoridagi eng yaqin butun songa yaxlitlash.

Masalan:

Dataset = 1000 rasm

Batch size = 32

bo'lsa:

$$1000 / 32 = 31.25$$

Yuqoriga yaxlitlanganda:

$$32 \text{ steps}$$

hosil bo'ladi.

Agar 10 epoch train qilinsa:

$$32 \times 10 = 320$$

ta optimizer update bajariladi.

Batafsil misol

Dataset:

128 ta rasm

Batch size:

16

Epoch:

5

Bitta epochdagi batchlar soni:

$$128/16=8 \quad 128/16=8 \quad 128/16=8$$

5 epochdagi umumiy step:

$$8 \times 5 = 40 \quad | \text{times } 5 = 40 \times 5 = 200$$

Model weightlari jami 200 marta yangilanadi.

Batch size katta bo'lsa

Masalan:

batch size = 64

Afzalliklari:

- GPU parallel ishlashdan yaxshiroq foydalanadi;
- training tezroq bo'lishi mumkin;
- gradient barqarorroq bo'ladi;
- loss egri chizig'i silliqroq ko'rinadi.

Kamchiliklari:

- ko'proq GPU VRAM kerak bo'ladi;
- gradient juda silliqlashib, generalization yomonlashishi mumkin;
- katta modelda CUDA out of memory xatosi chiqishi mumkin.

VRAM – Video Random Access Memory — GPU xotirasi.

CUDA out of memory — GPU xotirasi yetarli emasligi haqidagi xato.

Batch size kichik bo'lsa

Masalan:

batch size = 2

Afzalliklari:

- kamroq GPU xotirasi ishlatiladi;
- katta rasm yoki katta model bilan ishlash mumkin;
- gradientdagi tasodifiylik ba'zan generalizationni yaxshilaydi.

Kamchiliklari:

- training sekinroq bo'lishi mumkin;
- gradient shovqinliroq bo'ladi;
- loss qiymatlari ko'proq tebranadi;
- Batch Normalization barqarorligi pasayishi mumkin.

Generalization — modelning train datasetdan tashqari yangi ma'lumotlarda ham yaxshi ishlashi.

Batch Normalization — qatlam ichidagi qiymatlarni batch statistikasi asosida normallashtirish usuli.

Epoch soni kam bo'lsa

Epoch juda kam bo'lsa, model hali yetarli darajada o'rganmaydi.

Bu **underfitting** deyiladi.

Underfitting — model train datasetdagi qonuniyatlarni ham yetarlicha o'rgana olmagan holat.

Belgilar:

- train loss baland;
- val loss ham baland;
- precision va recall past;
- predictionlar beqaror.

Masalan, murakkab datasetda faqat 1 yoki 2 epoch training bajarilsa, model classlarni va bounding boxlarni yetarlicha o'rganmasligi mumkin.

Epoch soni juda ko'p bo'lsa

Epoch haddan tashqari ko'p bo'lsa, model train datasetni yodlab olishi mumkin.

Bu **overfitting** deyiladi.

Overfitting — model train datasetda juda yaxshi, yangi ma'lumotlarda esa yomon ishlaydigan holat.

Belgilar:

- train loss kamayishda davom etadi;
- val loss oshadi;
- train metric juda yuqori;
- validation yoki test metric pasayadi.

Batch size va epochning asosiy farqi

Xususiyat	Batch size	Epoch
Ma'nosi	Bir vaqtda ishlanadigan rasm soni	Butun datasetni to'liq ko'rish soni
Xotiraga ta'siri	Juda kuchli	To'g'ridan-to'g'ri kam
Training vaqtiga ta'siri	Har step tezligiga ta'sir qiladi	Umumiy training davomiyligiga ta'sir qiladi
Weight update soniga ta'siri	Bir epochdagi step sonini belgilaydi	Jami step sonini ko'paytiradi
Juda katta bo'lsa	Xotira yetmasligi mumkin	Overfitting bo'lishi mumkin
Juda kichik bo'lsa	Gradient shovqinli bo'ladi	Underfitting bo'lishi mumkin

YOLO uchun oddiy misollar

GPU xotirasi kam bo'lsa:

batch=2

yoki:

batch=1

ishlatiladi.

GPU xotirasi yetarli bo'lsa:

batch=8

yoki:

batch=16

ishlatilishi mumkin.

Sanity check uchun:

epochs=2

Pipeline tekshirish uchun:

epochs=5

To'liq tajriba uchun:

epochs=30

yoki:

epochs=50

tanlanishi mumkin.

Bu qiymatlar qat'iy qoida emas. Dataset hajmi, model o'lchami, GPU va natijalarga qarab belgilanadi.

3. Validation loss oshganda nima qilish kerak?

Validation loss nima?

Validation loss yoki **val loss** — modelning validation datasetdagi xatosi.

Validation dataset — training vaqtida weightlarni yangilash uchun ishlatilmaydigan, model sifatini nazorat qiluvchi alohida ma'lumotlar qismi.

Train loss model o'rganayotgan rasmlardagi xatoni ko'rsatadi.

Val loss esa model ko'rmagan yoki trainingga kiritilmagan rasmlardagi xatoni ko'rsatadi.

Training vaqtida odatda quyidagi holatlar kuzatiladi:

Yaxshi holat

Train loss kamayadi

Val loss kamayadi

Model train ma'lumotni o'rganmoqda va yangi ma'lumotda ham yaxshilanmoqda.

Overfitting holati

Train loss kamayadi

Val loss oshadi

Model train datasetni yodlab olmoqda, lekin validation ma'lumotda yomonlashmoqda.

Underfitting holati

Train loss baland

Val loss ham baland

Model hali vazifani yetarlicha o'rganmagan.

Data mismatch holati

Train loss ancha past

Val loss boshidan juda baland

Train va validation datasetlar orasida katta farq bo'lishi mumkin.

Data mismatch — train va validation ma'lumotlarining taqsimoti bir-biriga mos kelmasligi.

Masalan:

- train rasmlari kunduzgi;
- validation rasmlari kechasi;
- train rasmlari yuqori sifatli;
- validation rasmlari xira;
- train rasmlarida obyekt katta;
- validation rasmlarida obyekt juda kichik.

Birinchi navbatda loss trendini tekshirish kerak

Val loss bitta epochda biroz oshishi har doim muammo degani emas.

Masalan:

Epoch 1: val loss = 2.1

Epoch 2: val loss = 1.7

Epoch 3: val loss = 1.5

Epoch 4: val loss = 1.55

Epoch 5: val loss = 1.42

4-epochdagi kichik oshish tabiiy tebranish bo'lishi mumkin.

Muammo quyidagicha davomli trend bo'lsa yuzaga keladi:

Epoch 10: val loss = 0.80

Epoch 11: val loss = 0.86

Epoch 12: val loss = 0.94

Epoch 13: val loss = 1.05

Epoch 14: val loss = 1.18

Shu paytda val loss ketma-ket oshmoqda.

1. Early stopping ishlatish

Early stopping — validation natijasi yaxshilanmay qolsa trainingni avtomatik to'xtatish.

YOLO'da:

```
model.train(  
    epochs=100,  
    patience=10  
)
```

`patience=10` — validation metric 10 epoch davomida yaxshilanmasa training to'xtatiladi.

Patience — model yaxshilanishini kutish uchun berilgan epochlar soni.

Early stopping overfitting kuchayishidan oldin trainingni to'xtatishga yordam beradi.

Ultralytics odatda `best.pt` va `last.pt` fayllarini saqlaydi.

`best.pt` — validation natijasi eng yaxshi bo‘lgan epoch modeli.

`last.pt` — oxirgi epoch modeli.

Val loss keyinchalik oshgan bo‘lsa, `last.pt` emas, `best.pt` ishlatilishi kerak.

2. Epoch sonini kamaytirish

Agar model 50 epochdan keyin overfitting qila boshlasa, 100 epoch davom ettirish kerak emas.

Masalan:

1–20 epoch: val loss kamayadi

21–30 epoch: deyarli o‘zgarmaydi

31-epochdan keyin: val loss oshadi

Eng yaxshi checkpoint 20–30 epoch oralig‘ida bo‘lishi mumkin.

Shuning uchun epoch soni kamaytiriladi yoki early stopping ishlatiladi.

3. Datasetni ko‘paytirish

Overfittingning eng samarali yechimlaridan biri — train datasetni ko‘paytirish.

Ko‘proq ma’lumot modelga turli holatlarni o‘rganishga yordam beradi.

Masalan, avtomobil datasetiga quyidagilar qo‘shiladi:

- turli rangdagi avtomobillar;
- turli ob-havo;
- turli kamera burchaklari;
- kecha va kunduz;

- turli fonlar;
- qisman yopilgan avtomobillar;
- yaqin va uzoq obyektlar.

Faqat bir xil sahnadagi deyarli takroriy rasmlarni qo'shish yetarli emas. Yangi ma'lumot haqiqiy xilma-xillik olib kelishi kerak.

4. Data augmentation ishlatish

Data augmentation — train rasmlarini sun'iy o'zgartirib, modelga ko'proq xilma-xillik ko'rsatish.

Object detection uchun quyidagilar ishlatiladi:

- horizontal flip;
- scale;
- crop;
- brightness;
- contrast;
- blur;
- noise;
- mosaic;
- mixup.

Scale — rasm yoki obyekt o'lchamini o'zgartirish.

Contrast — och va to'q qismlar orasidagi farq.

Mosaic — to'rtta rasmni bitta katta rasmga birlashtiruvchi augmentation.

MixUp — ikki rasm va ularning label ma'lumotlarini aralashtirish usuli.

Augmentation faqat train datasetga qo'llanishi kerak. Validation dataset real holatda qolishi kerak.

Validationga kuchli random augmentation qo'llansa, metriclarni ishonchli taqqoslash qiyinlashadi.

5. Modelni kichraytirish

Katta model kichik datasetni tez yodlab olishi mumkin.

Masalan:

YOLOv8x

juda kichik datasetda overfitting qilishi ehtimoli yuqori.

Shunday holatda:

YOLOv8n

YOLOv8s

YOLOv8m

kabi kichikroq model tanlanishi mumkin.

Model parametrlari soni kamayganda modelning train ma'lumotni yodlab olish qobiliyati ham kamayadi.

6. Regularization ishlatish

Regularization — modelning haddan tashqari murakkablashishini va train datasetni yodlashini cheklash usullari.

Asosiy usullar:

- weight decay;
- dropout;
- data augmentation;
- label smoothing.

Weight decay

Weightlarning juda katta qiymatlarga o'sishini cheklaydi.

Dropout

Training vaqtida ayrim neyronlarni tasodifiy o'chirib turadi.

Dropout — modelni bitta yo'lga haddan tashqari bog'lanib qolishdan saqlovchi usul.

YOLO arxitekturasida dropout har doim asosiy sozlama sifatida ishlatilmasligi mumkin, lekin umumiy neural networklarda keng qo'llanadi.

Label smoothing

Modelning target classga 100 foiz mutlaq ishonch bilan yopishib qolishini kamaytiradi.

Label smoothing — target qiymatlarni biroz yumshatish.

7. Learning rate'ni kamaytirish

Val loss keskin tebranayotgan bo'lsa, learning rate katta bo'lishi mumkin.

Masalan:

$$lr_0 = 0.01$$

o'rniga:

$$lr_0 = 0.001$$

sinab ko'rilishi mumkin.

Katta learning rate weightlarni optimal nuqta atrofida sakratishi mumkin.

Learning rate scheduler ishlatish ham mumkin.

Scheduler — training davomida learning rate'ni avtomatik o'zgartiruvchi mexanizm.

Masalan, dastlab learning rate katta, keyin asta-sekin kichrayadi.

8. Train va validation splitni tekshirish

Val lossning oshishiga noto'g'ri dataset bo'linishi ham sabab bo'lishi mumkin.

Quyidagilar tekshirilishi kerak:

- classlar train va validationda mavjudmi;
- validation juda kichik emasmi;
- train va validation rasmlari bir xil manbadanmi;
- validationda annotatsiya xatosi yo'qmi;
- video kadrlari noto'g'ri ajratilmaganmi;
- duplicate rasmlar mavjudmi.

Duplicate — takroriy yoki bir xil fayl.

Video datasetda yonma-yon kadrlarni tasodifiy train va validationga bo'lish data leakage keltirib chiqarishi mumkin.

Data leakage — validation yoki test ma'lumotining trainingga bilvosita kirib qolishi.

Bunday holatda metricalar dastlab yolg'on yuqori chiqishi mumkin.

9. Annotationlarni tekshirish

Validation loss baland bo'lsa, validation annotationlari xato bo'lishi mumkin.

Tekshiriladigan holatlar:

- class ID noto'g'ri;
- bounding box obyektini to'liq qamramagan;
- rasmda obyekt bor, label yo'q;
- bir xil class turlicha belgilangan;
- box rasm chegarasidan tashqariga chiqqan;
- width yoki height noto'g'ri normallashtirilgan.

Masalan, aslida truck bo'lgan obyekt car deb belgilansa, model to'g'ri prediction qilgan taqdirda ham loss oshishi mumkin.

10. Class imbalance'ni tekshirish

Class imbalance — classlar sonining keskin teng bo'lmasligi.

Masalan:

car: 5000 obyekt
truck: 200 obyekt
bus: 50 obyekt

Model car classini yaxshi, bus classini yomon o'rganishi mumkin.

Yechimlar:

- kam class uchun ko'proq rasm yig'ish;
- kam classli rasmlarni ko'proq sampling qilish;
- mos augmentation ishlatish;
- classlar bo'yicha natijani alohida tekshirish.

Sampling — ma'lumotlarni tanlash usuli.

Val loss oshganda bajariladigan tartib

Quyidagi ketma-ketlik bo'yicha tekshirish kerak:

1. Loss faqat bir marta oshdimi yoki trend davomlimi?
2. best.pt qaysi epochda saqlangan?
3. Train loss va val loss orasidagi farq qanday?
4. Annotationlar to'g'rimi?
5. Train/val split to'g'rimi?
6. Dataset yetarlicha xilma-xilmi?
7. Augmentation mavjudmi?
8. Model dataset uchun juda kattami?
9. Learning rate baland emasmi?
10. Early stopping ishlatilganmi?

4. Precision va Recall — qachon qaysi biri muhimroq?

Precision va recall modelning xatolarini ikki xil nuqtai nazardan baholaydi.

Ularni tushunish uchun avval to'rtta asosiy holatni bilish kerak:

- True Positive;
- False Positive;
- False Negative;
- True Negative.

True Positive

True Positive — TP — obyekt mavjud va model uni to'g'ri topgan.

Masalan, rasmda avtomobil mavjud va model car boxini to'g'ri chizgan.

False Positive

False Positive — FP — obyekt mavjud emas, lekin model mavjud deb topgan.

Masalan, yo‘ldagi soyani model avtomobil deb belgilagan.

Bu **false alarm**, ya’ni yolg‘on ogohlantirish deyiladi.

False Negative

False Negative — FN — obyekt mavjud, lekin model uni topmagan.

Masalan, rasmda piyoda mavjud, model esa hech qanday box chiqarmagan.

Bu obyektни o‘tkazib yuborish hisoblanadi.

True Negative

True Negative — TN — obyekt mavjud emas va model ham obyekt yo‘q deb to‘g‘ri aniqlagan.

Object detection confusion matrixda background bilan bog‘liq hisoblar mavjud bo‘lsa-da, TN sonini an’anaviy classificationdagi kabi to‘liq hisoblash qiyinroq. Chunki rasmda obyekt bo‘lmagan fazoviy joylar soni juda katta bo‘ladi.

Precision formulasi

$$Precision = \frac{TP}{TP + FP}$$

Precision model topgan obyektlarning qanchasi haqiqatan to‘g‘ri ekanini ko‘rsatadi.

Masalan:

Model 100 ta obyekt topdi

80 tasi to'g'ri

20 tasi noto'g'ri

bo'lsa:

$$Precision = \frac{80}{80+20} = 0.80$$
$$Precision = \frac{80}{80+20} = 0.80$$

ya'ni:

$$Precision = 80\%$$

Precision qachon muhim?

False Positive xatosi qimmat yoki zararli bo'lgan vazifalarda precision muhimroq.

Spam email aniqlash

Oddiy emailni spam deb noto'g'ri o'chirish zararli.

Bu yerda False Positive kamayishi kerak.

Yuz orqali kirish tizimi

Begona odamni ruxsatli shaxs deb qabul qilish xavfli.

Precision yuqori bo'lishi kerak.

Avtomatik jarima tizimi

Qoidani buzmagani avtomobilga jarima yozish noto'g'ri.

False Positive kamaytirilishi kerak.

Zavodda mahsulotni brak deb ajratish

Sogʻlom mahsulotni nuqsonli deb tashlab yuborish moddiy zarar keltiradi.

Precision muhim boʻladi.

Robotik tizim

Robot mavjud boʻlmagan obyektни mavjud deb qabul qilib, notoʻgʻri harakat qilishi mumkin.

Precision yuqori boʻlishi kerak.

Recall formulasi

$$Recall = \frac{TP}{TP + FN}$$

Recall haqiqiy obyektlarning qanchasini model topa olganini koʻrsatadi.

Masalan:

Rasm va videolarda 100 ta haqiqiy obyekt bor
Model 90 tasini topdi
10 tasini oʻtkazib yubordi

boʻlsa:

$$Recall = \frac{90}{90 + 10} = 0.90$$

yaʼni:

$$Recall = 90\%$$

Recall qachon muhim?

False Negative xatosi xavfli bo'lgan vazifalarda recall muhimroq.

Kasallik aniqlash

Kasal bemorni sog'lom deb o'tkazib yuborish xavfli.

Shubhali holatlarni ko'proq topish muhim.

Yong'in yoki tutun aniqlash

Haqiqiy yong'inni topmay qolish katta zarar keltiradi.

False Negative juda xavfli.

Piyoda aniqlash

Avtonom avtomobil yo'ldagi piyodani topmay qolsa, og'ir oqibat yuz berishi mumkin.

Recall yuqori bo'lishi kerak.

Xavfsizlik kamerasi

Qurol yoki shubhali obyektni o'tkazib yuborish xavfli.

Recall muhimroq.

Nuqson aniqlash

Kritik detal ichidagi nuqsonni topmay qolish mahsulotning buzilishiga olib kelishi mumkin.

Recall yuqori bo'lishi kerak.

Precision va recall orasidagi muvozanat

Ko'pincha precision va recall orasida trade-off mavjud.

Trade-off — bitta ko'rsatkich yaxshilanganda boshqasining yomonlashishi mumkin bo'lgan muvozanat.

Object detectionda confidence threshold o'zgartirilganda bu holat ko'rinadi.

Confidence threshold pasaytirilsa

Masalan:

$conf = 0.10$

Model ko'proq prediction chiqaradi.

Natija:

- ko'proq haqiqiy obyekt topilishi mumkin;
- False Negative kamayadi;
- recall oshadi;
- noto'g'ri predictionlar ham ko'payishi mumkin;
- False Positive ortadi;
- precision pasayishi mumkin.

Confidence threshold oshirilsa

Masalan:

$conf = 0.80$

Model faqat juda ishonchli predictionlarni qoldiradi.

Natija:

- noto'g'ri prediction kamayadi;

- precision oshishi mumkin;
- ayrim haqiqiy obyektlar chiqarilmaydi;
- False Negative ortadi;
- recall pasayishi mumkin.

Misol

Haqiqiy obyektlar soni:

100 ta

Past threshold

TP = 95

FP = 30

FN = 5

Precision:

$$95/(95+30)=0.76 \quad 95/(95+30)=0.76 \quad 95/(95+30)=0.76$$

Recall:

$$95/(95+5)=0.95 \quad 95/(95+5)=0.95 \quad 95/(95+5)=0.95$$

Natija:

Precision = 76%

Recall = 95%

Yuqori threshold

TP = 75

FP = 5

FN = 25

Precision:

$$75/(75+5)=0.9375 \quad 75/(75+5)=0.9375 \quad 75/(75+5)=0.9375$$

Recall:

$$75/(75+25)=0.75 \quad 75/(75+25)=0.75 \quad 75/(75+25)=0.75$$

Natija:

Precision = 93.75%

Recall = 75%

Yuqori threshold precisionni oshirgan, recallni esa kamaytirgan.

F1-score

Precision va recallni bitta qiymatda birlashtirish uchun **F1-score** ishlatiladi.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall} \quad F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

Masalan:

$$Precision = 0.80 \quad Precision = 0.80 \quad Precision = 0.80$$

$$Recall = 0.90 \quad Recall = 0.90 \quad Recall = 0.90$$

bo'lsa:

$$F1 = 2 \times \frac{0.80 \times 0.90}{0.80 + 0.90} \quad F1 = 2 \times \frac{0.80 \times 0.90}{0.80 + 0.90} \quad F1 \approx 0.847 \quad F1 \approx 0.847$$

F1-score taxminan 84.7 foiz bo'ladi.

F1-score precision va recall ikkalasi ham muhim bo'lgan vazifalarda foydali.

Harmonic mean ishlatilgani sababli, precision yoki recall juda past bo'lsa, F1-score ham keskin pasayadi.

Harmonic mean — kichik qiymatga ko'proq ta'sirchan o'rtacha hisoblash usuli.

5. Confusion matrix nima ko'rsatadi?

Confusion matrix ta'rifi

Confusion matrix — modelning haqiqiy classlar bilan prediction qilingan classlarni solishtiruvchi jadval.

Confusion — chalkashlik.

Matrix — qator va ustunlardan tashkil topgan sonlar jadvali.

Confusion matrix model qaysi classlarni to'g'ri topgani va qaysi classlarni bir-biri bilan adashtirayotganini ko'rsatadi.

Binary classification confusion matrix

Ikki classli vazifada confusion matrix quyidagicha bo'ladi:

	Prediction Positive	Prediction Negative
Actual Positive	TP	FN
Actual Negative	FP	TN

Actual — haqiqiy class.

Prediction — model bashorati.

Masalan:

	Model: kasal	Model: sog'lom
Haqiqiy kasal	90	10
Haqiqiy sog'lom	5	95

Bu yerda:

TP = 90

FN = 10

FP = 5

TN = 95

Precision:

$$90/(90+5)=0.94790/(90+5)=0.94790/(90+5)=0.947$$

Recall:

$$90/(90+10)=0.9090/(90+10)=0.9090/(90+10)=0.90$$

Multiclass confusion matrix

Bir nechta class mavjud bo'lsa, matrix kattalashadi.

Masalan, uchta class:

car

bus

truck

Confusion matrix:

Haqiqiy \ Prediction	car	bus	truck
car	80	5	15
bus	4	90	6
truck	20	8	72

Diagonal qiymatlar

Chap yuqoridan o'ng pastga tushuvchi qiymatlar diagonal deyiladi:

80, 90, 72

Bular to'g'ri predictionlar.

- 80 ta car to'g'ri car deb topilgan;
- 90 ta bus to'g'ri bus deb topilgan;
- 72 ta truck to'g'ri truck deb topilgan.

Diagonal qiymatlar katta bo'lishi yaxshi.

Diagonaldan tashqari qiymatlar

Masalan:

car → truck = 15

15 ta haqiqiy car model tomonidan truck deb topilgan.

truck → car = 20

20 ta haqiqiy truck model tomonidan car deb topilgan.

Bu model car va truckni ko'p adashtirayotganini ko'rsatadi.

Sabablar:

- avtomobil va yuk mashinasi o'xshash ko'rinishi;
- rasmlar sifati pastligi;
- obyektning faqat bir qismi ko'rinishi;
- annotation xatolari;
- train datasetda truck rasmlari kamligi;
- uzoqdagi truck kichik ko'rinishi.

Object detection confusion matrix

Object detectiondagi confusion matrix oddiy classification matrixdan biroz murakkabroq.

Chunki model faqat classni emas, obyekt joylashuvini ham to'g'ri topishi kerak.

Prediction True Positive bo'lishi uchun:

1. class to'g'ri bo'lishi;
2. bounding box haqiqiy box bilan yetarli darajada ustma-ust tushishi kerak.

Ustma-ust tushish **IoU** bilan o'lchanadi.

IoU – Intersection over Union — ikki bounding boxning kesishgan maydonini ularning birlashgan maydoniga bo'lish.

$$IoU = \frac{\text{Intersection}}{\text{Union}}$$

Masalan:

$$IoU = 0.80$$

bo'lsa, prediction box va haqiqiy box juda yaxshi mos tushgan.

$$IoU = 0.10$$

bo'lsa, boxlar deyarli mos kelmagan.

Model classni to'g'ri topgan bo'lsa ham, box juda noto'g'ri joylashsa, detection noto'g'ri hisoblanishi mumkin.

Background qatori va ustuni

YOLO confusion matrixida ko'pincha **background** qatori yoki ustuni ham mavjud bo'ladi.

Background — target obyekt bo'lmagan fon.

Actual class → background

Masalan, haqiqiy person background sifatida qolsa, model personni topmagan bo‘ladi.

Bu False Negative hisoblanadi.

Background → prediction class

Masalan, haqiqiy obyekt bo‘lmagan joyda model car topgan bo‘lsa, bu False Positive hisoblanadi.

Shuning uchun object detection confusion matrix:

- classlarni bir-biri bilan adashtirish;
- topilmay qolgan obyektlar;
- ortiqcha predictionlar

haqida ma’lumot beradi.

Confusion matrixdan nimalarni bilish mumkin?

1. Qaysi class yaxshi o‘rganilgan?

Diagonal qiymati katta class yaxshi o‘rganilgan bo‘lishi mumkin.

2. Qaysi classlar adashtirilmoqda?

Diagonaldan tashqari katta qiymatlar qaysi classlar bir-biriga aralashayotganini ko‘rsatadi.

3. Qaysi class ko‘p o‘tkazib yuborilmoqda?

Classdan backgroundga ketgan qiymatlar ko‘p bo‘lsa, model shu classni topmay qolmoqda.

Bu recall muammosini ko‘rsatadi.

4. Qaysi class ortiqcha topilmoqda?

Backgrounddan classga kelgan qiymatlar ko'p bo'lsa, model mavjud bo'lmagan obyektlarni shu class deb topmoqda.

Bu precision muammosini ko'rsatadi.

5. Dataset imbalance mavjudmi?

Ayrim classlarda umumiy son juda katta, boshqalarida juda kichik bo'lsa, confusion matrixdagi qatorlar ham keskin farq qiladi.

Oddiy va normalized confusion matrix

Oddiy confusion matrix

Haqiqiy sonlarni ko'rsatadi.

Masalan:

car → car = 500

truck → truck = 40

Bu classlar sonining farqini ham aks ettiradi.

Normalized confusion matrix

Normalized confusion matrix qiymatlarni ulush yoki foiz ko'rinishida ko'rsatadi.

Masalan:

car → car = 0.90

car → truck = 0.10

Bu haqiqiy car obyektlarining:

- 90 foizi car deb;
- 10 foizi truck deb

topilganini bildiradi.

Normalized matrix classlar soni teng bo'lmaganda taqqoslash uchun qulayroq.

Confusion matrixni tahlil qilish misoli

Quyidagi natija mavjud bo'lsin:

Haqiqiy \ Prediction	car	bus	truck	background
car	90	2	6	2
bus	3	70	5	22
truck	15	5	65	15
background	20	4	10	—

Tahlil:

- car classining 90 tasi to'g'ri topilgan;
- bus classining 22 tasi umuman topilmagan;
- truckning 15 tasi car deb adashtirilgan;
- model fon ichida 20 marta noto'g'ri car topgan.

Xulosa:

- car recall yaxshi;
- bus recall past;
- truck va car orasida class chalkashligi mavjud;
- car precisioniga backgrounddagi noto'g'ri predictionlar salbiy ta'sir qilmoqda.

Yechim sifatida:

- bus rasmlarini ko'paytirish;
- uzoqdagi bus rasmlarini qo'shish;
- car va truckni ajratadigan xilma-xil misollar yig'ish;
- annotationlarni tekshirish;
- confidence thresholdni sozlash;

- train dataset balansini yaxshilash

kerak bo'лади.

Umumiy bog'lanish

Barcha tushunchalar bitta training jarayonida quyidagicha bog'lanadi:

Dataset batchlarga bo'linadi

→ har batchda forward pass bajariladi

→ loss hisoblanadi

→ backward pass bajariladi

→ weightlar yangilanadi

→ barcha batchlar tugasa 1 epoch tugaydi

→ validation datasetda val loss hisoblanadi

→ precision va recall aniqlanadi

→ confusion matrix class xatolarini ko'rsatadi

Forward pass prediction hosil qiladi. Backward pass xatodan foydalanib modelni o'rgatadi. Batch size bir vaqtda nechta rasm qayta ishlanishini belgilaydi. Epoch butun dataset necha marta o'rganilishini bildiradi. Val loss modelning yangi ma'lumotda qanday ishlayotganini nazorat qiladi. Precision noto'g'ri ogohlantirishlarni, recall esa o'tkazib yuborilgan obyektlarni tahlil qiladi. Confusion matrix esa qaysi classlar to'g'ri topilgani va qaysilari o'zaro chalkashayotganini batafsil ko'rsatadi.